



Web Programming

MERN - MySQL - Building Full Stack
Apps with SQL Database

Lecture 18

Mahhek Tahir



Why MySQL instead of MongoDB

Structured data (tables, rows)

Strong relationships (foreign keys)

Widely used in industry

Easier to understand data integrity



Comparision

Feature	MySQL (SQL)	MongoDB (NoSQL)
Structure	Tables	Collections
Data Format	Rows	JSON Documents
Schema	Fixed	Flexible
Relations	Strong	Weak



Command

```
npm install express mysql2 cors dotenv
```

Database & tables are usually created manually (validations can be added here like constraints)

- Using:
 - MySQL Workbench
 - phpMyAdmin
 - SQL console

Node.js is used to access and interact

- Insert data
- Fetch data
- Update / delete



Flow

1. Create database (MySQL)
2. Create tables (MySQL)
3. Connect using Node.js
4. Run queries from backend

mysql -u root -p // in terminal after installing mysql

```
CREATE DATABASE test_db;
```

```
USE test_db;
```

```
CREATE TABLE posts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(200),  
  user_id INT,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

config/db.js

```
const mysql = require("mysql2");

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "your_password",
  database: "test_db"
});

db.connect((err) => {
  if (err) {
    console.error("Connection failed:", err);
  } else {
    console.log("MySQL Connected");
  }
});

module.exports = db;
```

index.js

```
const express = require("express");
const cors = require("cors");
const db = require("../config/db");

const app = express();

app.use(cors());
app.use(express.json());

app.get("/users", (req, res) => {
  db.query("SELECT * FROM users", (err, result) => {
    if (err) return res.send(err);
    res.json(result);
  });
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Insert Data Example



```
app.post("/users", (req, res) => {  
  const { name, email } = req.body;  
  
  const sql = "INSERT INTO users (name, email) VALUES (?, ?)";  
  
  db.query(sql, [name, email], (err, result) => {  
    if (err) return res.send(err);  
    res.send("User added");  
  });  
});
```


Update (put)

```
app.put("/users/:id", (req, res) => {  
  const { name, email } = req.body;  
  const { id } = req.params;  
  
  const sql = "UPDATE users SET name = ?, email = ? WHERE id = ?";  
  
  db.query(sql, [name, email, id], (err, result) => {  
    if (err) return res.status(500).send(err);  
    res.send("User updated ✅");  
  });  
});
```



Delete

```
app.delete("/users/:id", (req, res) => {  
  const { id } = req.params;  
  
  db.query("DELETE FROM users WHERE id = ?", [id], (err, result) => {  
    if (err) return res.status(500).send(err);  
    res.send("User deleted ✖");  
  });  
});
```



Validations

Validations are done in Node.js (NOT in a model like Mongoose).

```
app.post("/users", (req, res) => {  
  const { name, email, password, age } = req.body;  
  
  if (!name || !email || !password) {  
    return res.send("All fields required");  
  }  
  
  if (name.length < 3) {  
    return res.send("Name must be at least 3 characters");  
  }  
  
  if (password.length < 6) {  
    return res.send("Password too short");  
  }  
  
  if (age < 18 || age > 60) {  
    return res.send("Age must be between 18 and 60");  
  }  
});
```

Schema level validation

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  age INT CHECK (age >= 18 AND age <= 60)  
);
```



Defaults

```
const role = req.body.role || "user";
```

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  role VARCHAR(50) DEFAULT 'user',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Relations

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

```
CREATE TABLE posts (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(100),  
    user_id INT,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Joins

```
app.get("/posts", (req, res) => {  
  const sql = `  
    SELECT users.name, posts.title  
    FROM users  
    INNER JOIN posts ON users.id = posts.user_id  
  `;  
  
  db.query(sql, (err, result) => {  
    if (err) return res.send(err);  
    res.json(result);  
  });  
});
```


Connecting Flow

react: `fetch("/users") ->` node: `app.get("/users", ...) ->`
mysql: `SELECT * FROM users; ->` `res.json(result); ->`
react: `setUsers(data);`

```
useEffect(() => {  
  fetch("http://localhost:5000/users")  
    .then(res => res.json())  
    .then(data => setUsers(data));  
}, []);
```

```
app.get("/users", (req, res) => {  
  db.query("SELECT * FROM users", (err, result) => {  
    if (err) return res.send(err);  
    res.json(result);  
  });  
});
```